

Lecture 25

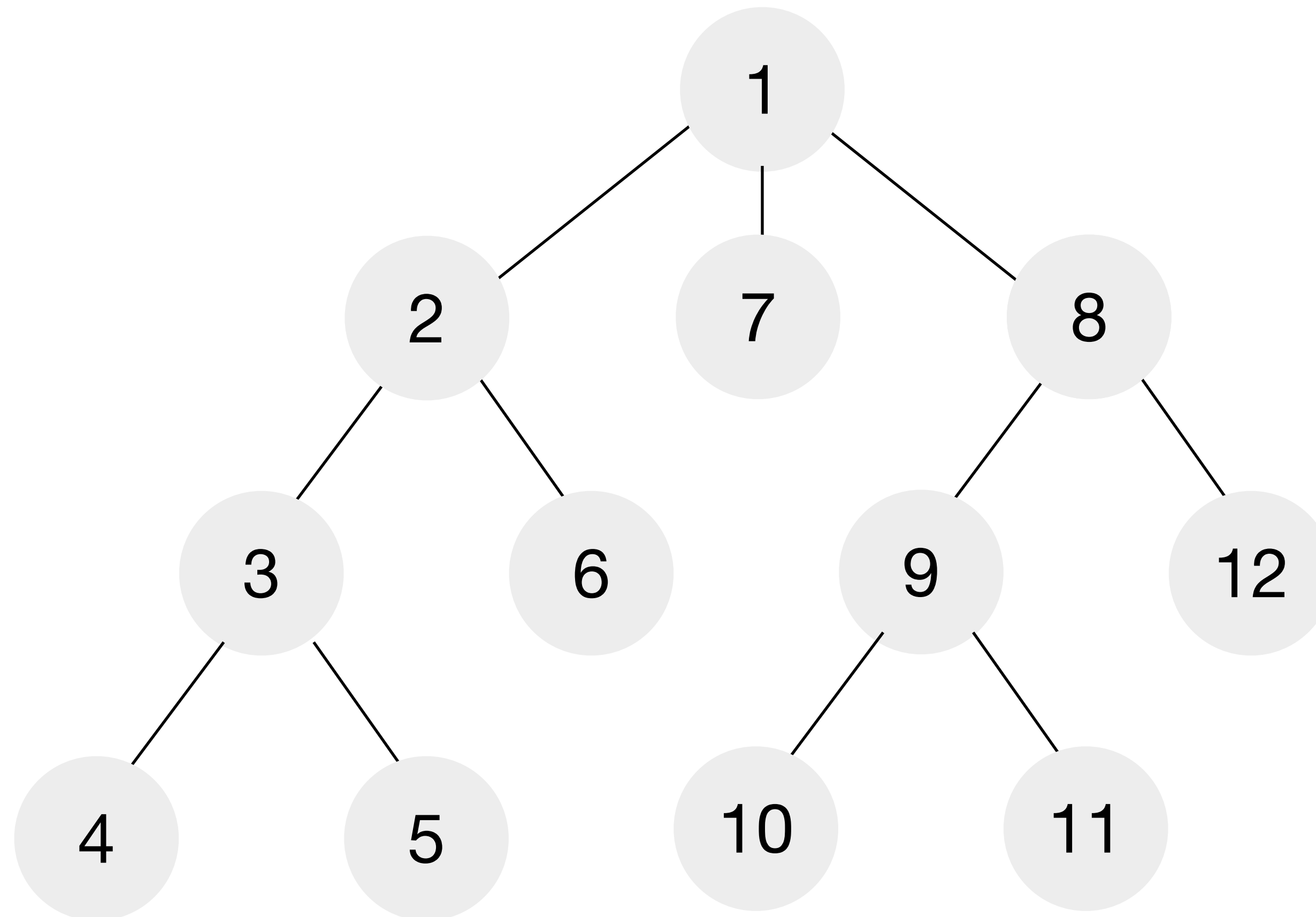
Graphs

Tree is a type of graph

- Graph: a data structure that consists of vertices (nodes) and edges

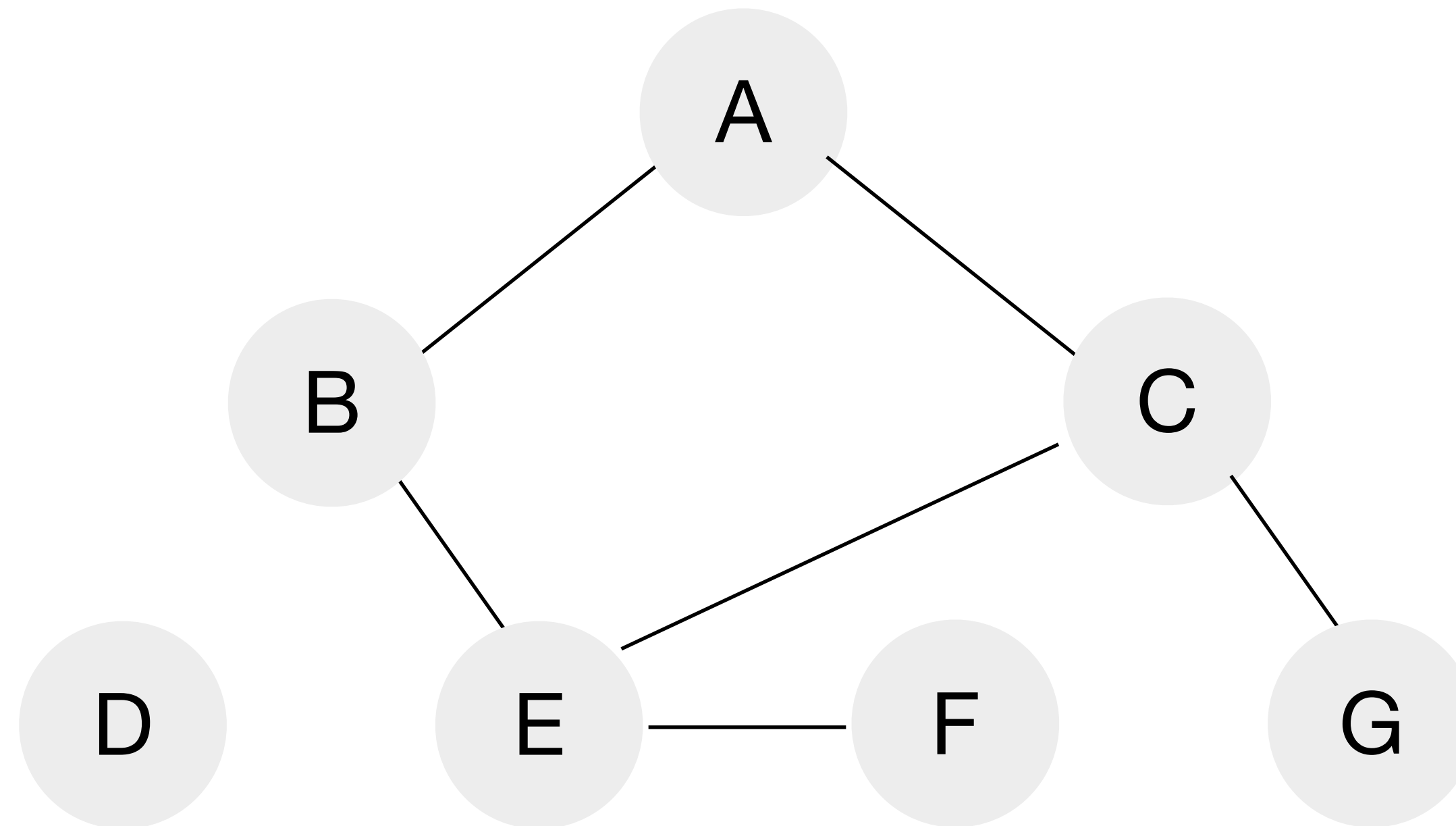
Tree is a type of graph

- Graph: a data structure that consists of vertices (nodes) and edges



Tree is a type of graph

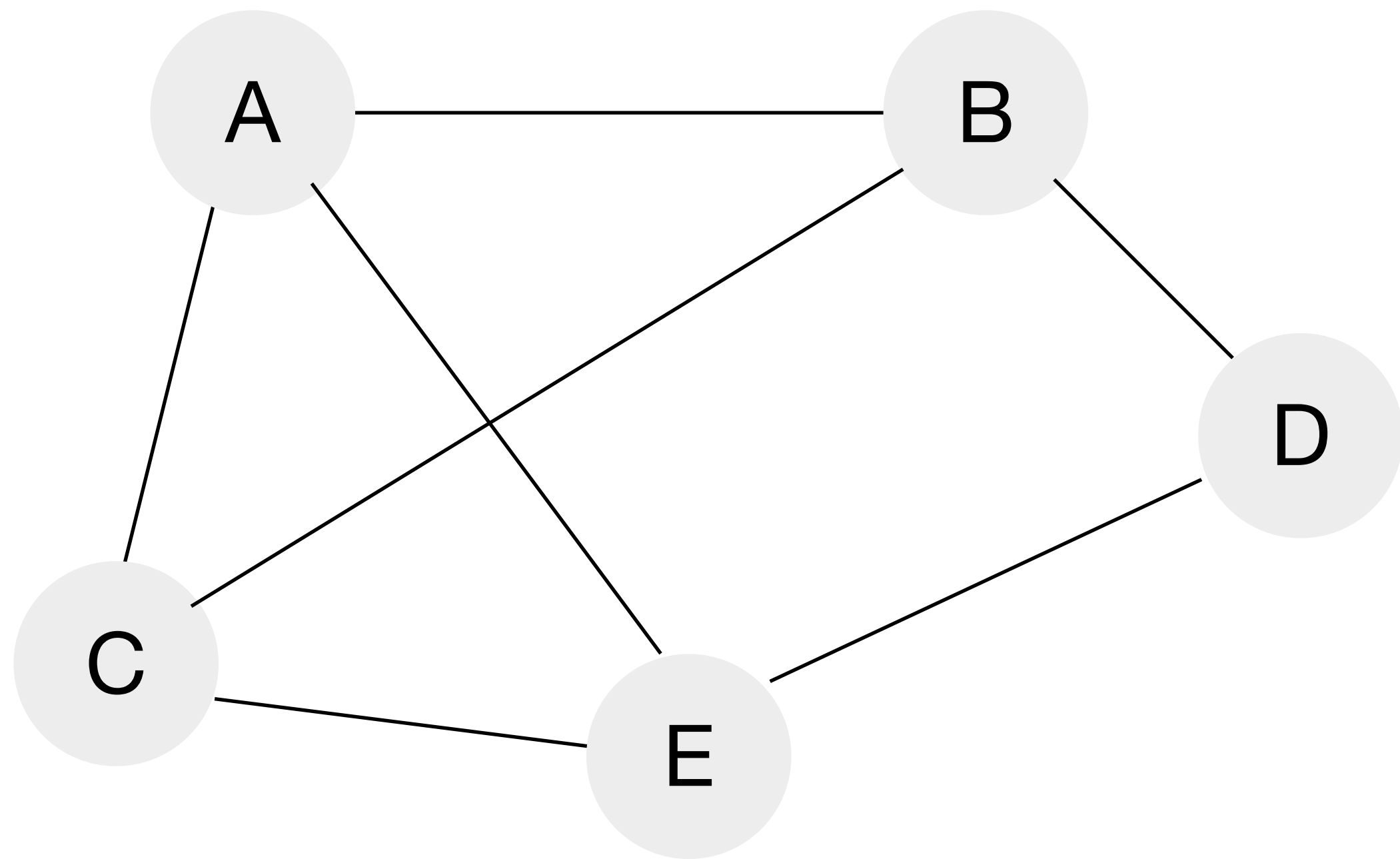
- Graph: a data structure that consists of vertices (nodes) and edges



- What are some examples of datasets that might suit this structure?
 - **class[Buzz]: graph_examples**

Adjacency matrix

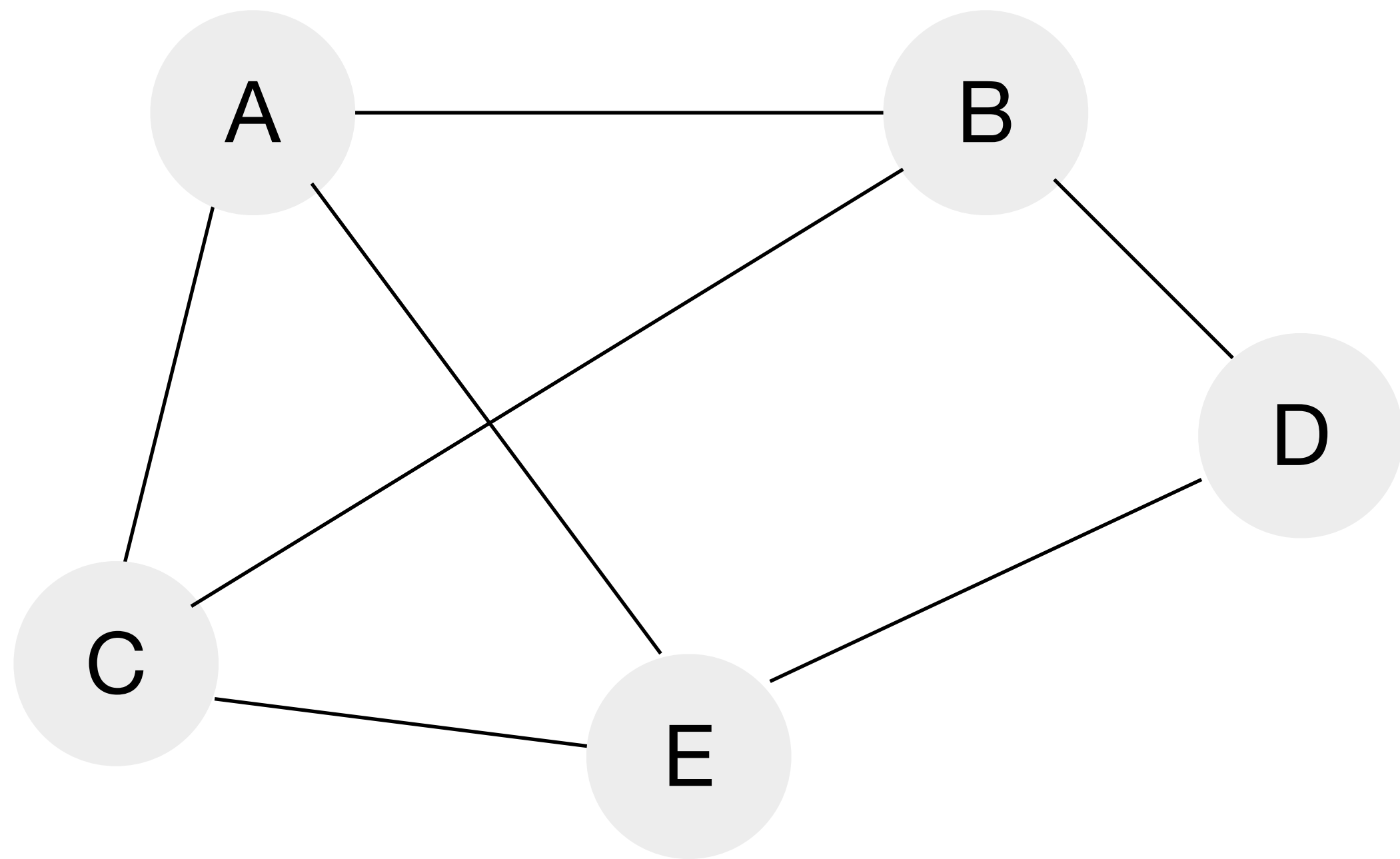
- A simple way of encoding the connectivity of a graph



	A	B	C	D	E
A	0	1	1	0	1
B	1	0	1	1	0
C	1	1	0	0	1
D	0	1	0	0	1
E	1	0	1	1	0

Adjacency list

- An alternative way of encoding the connectivity of a graph



A → B → C → E

B → A → C → D

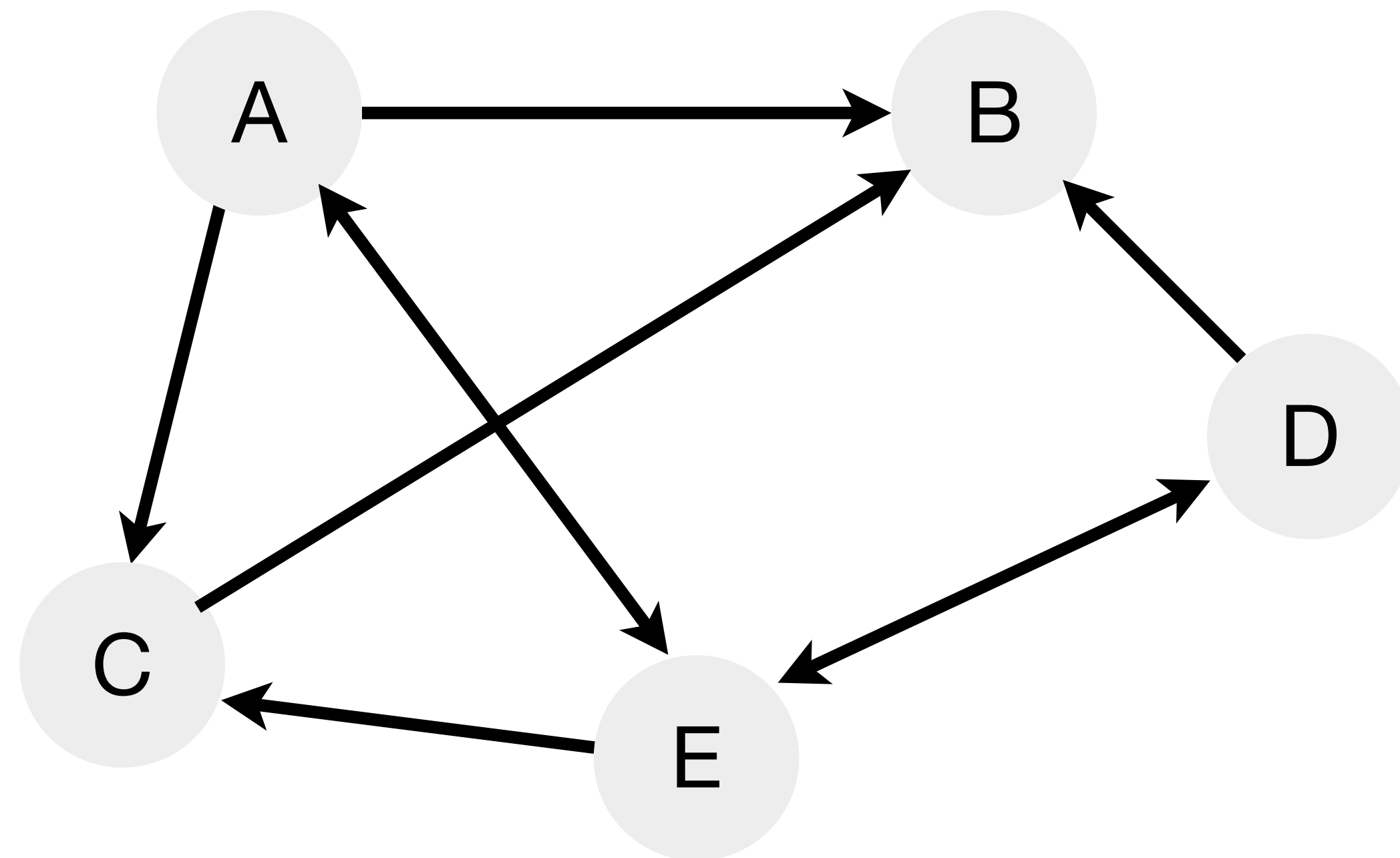
C → A → B → E

D → B → E

E → A → C → D

Directed graphs

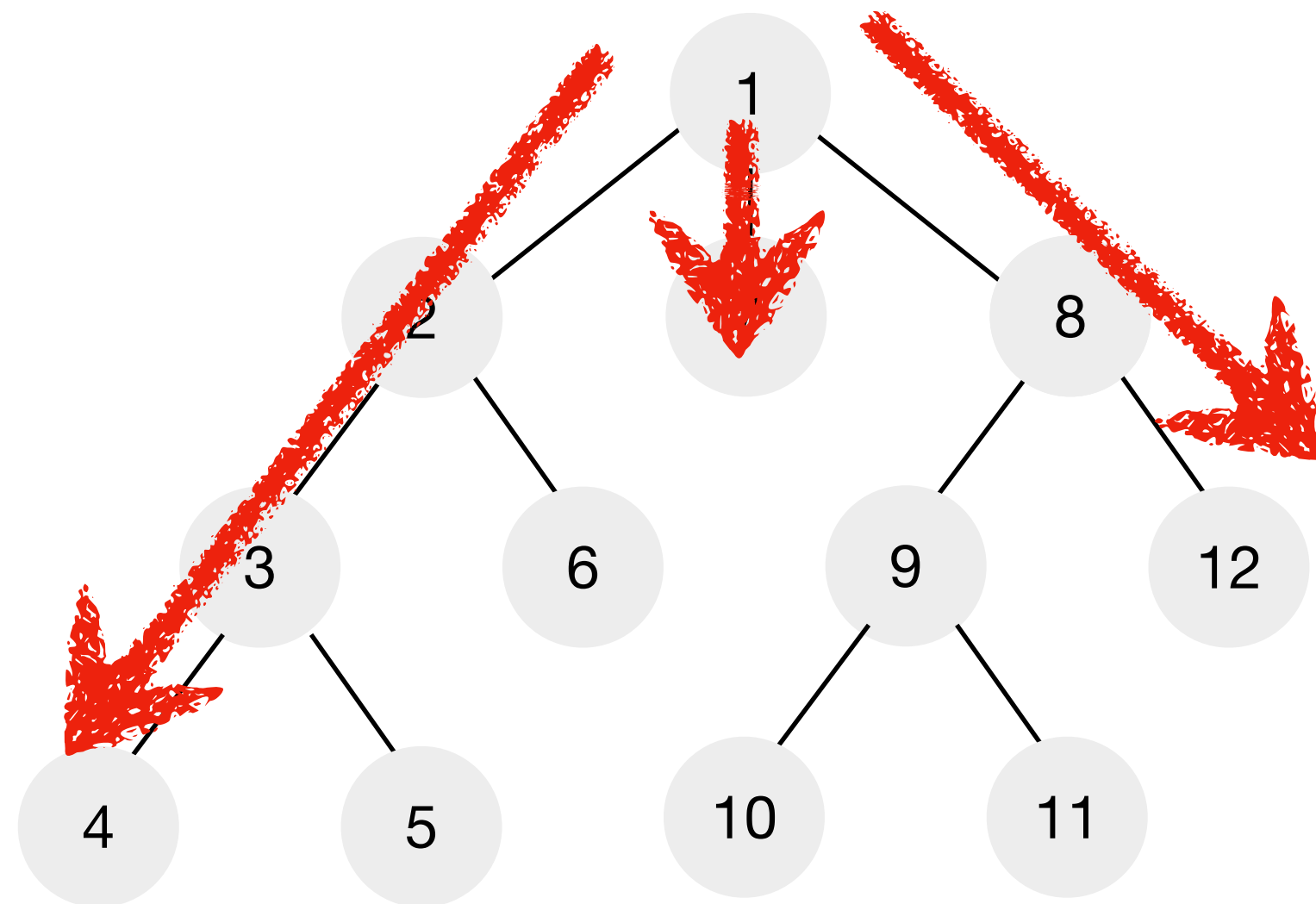
- Edges might have an inherent directionality, for example in social networks



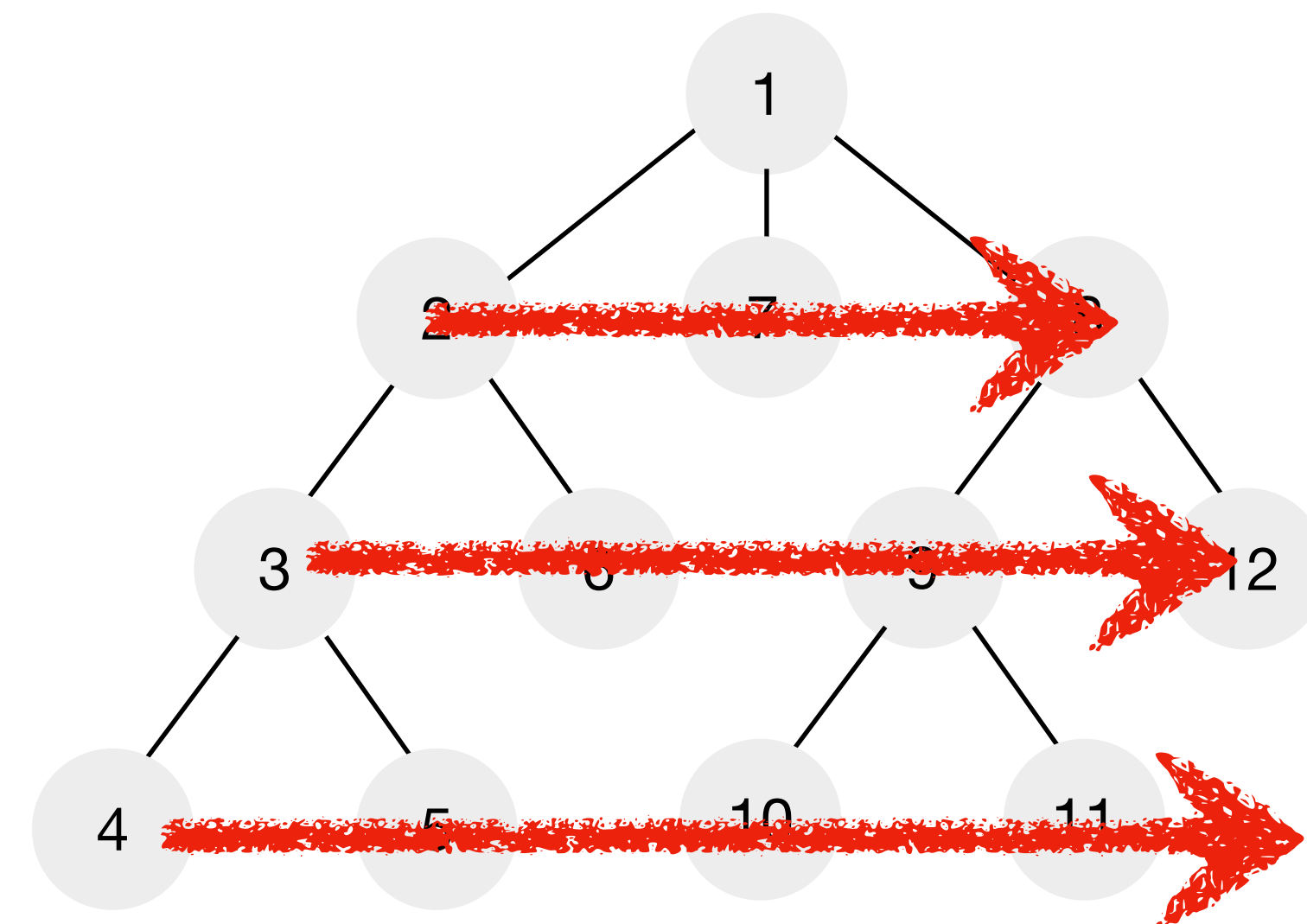
Graph Traversal

- One of the most common operations performed on a graph is traversal.
- This can be done depth first, or breadth first

Depth First



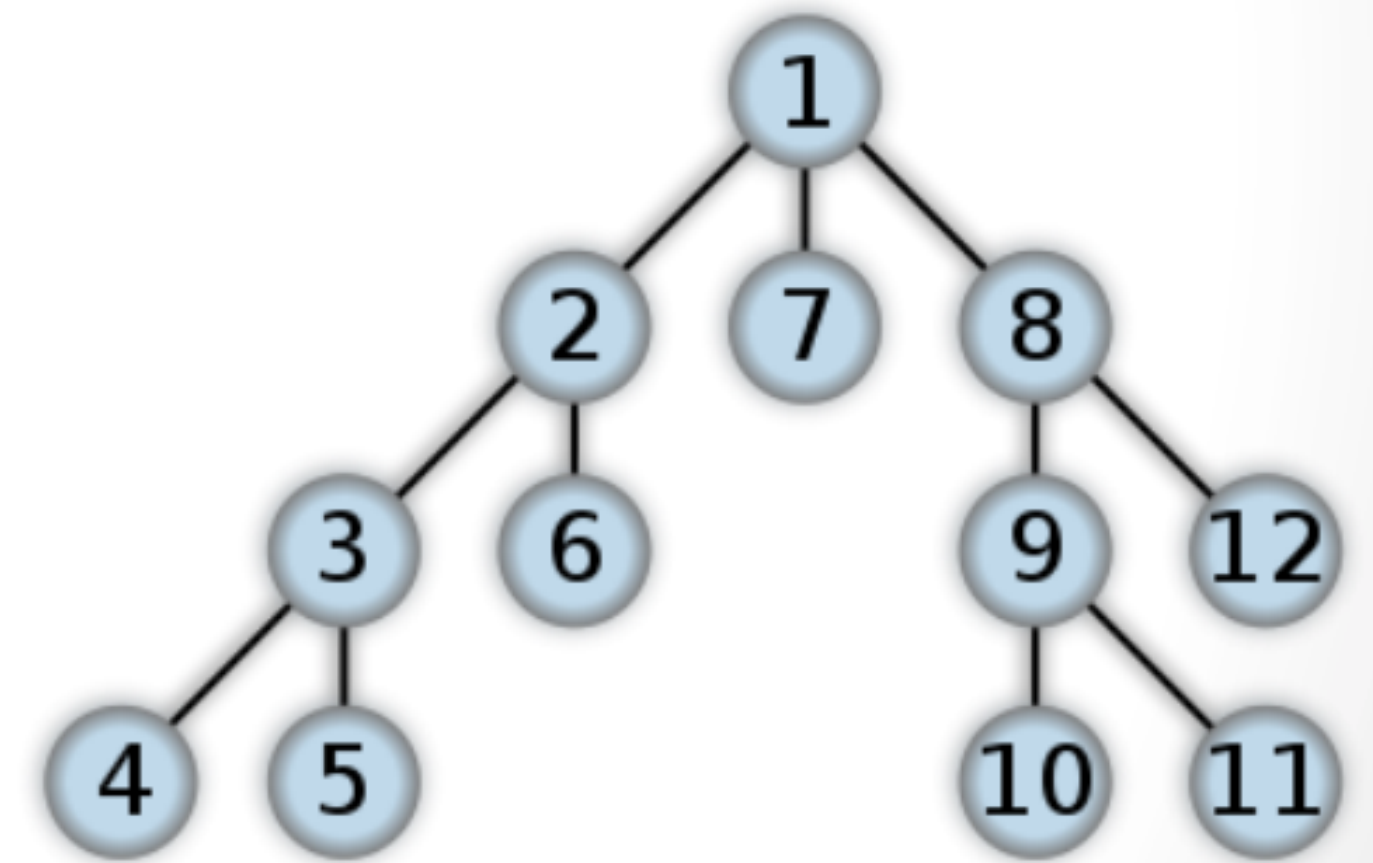
Breadth First



DFS uses a stack

```
/* G is the graph (or tree) to search  
v is the vertex where the search starts */
```

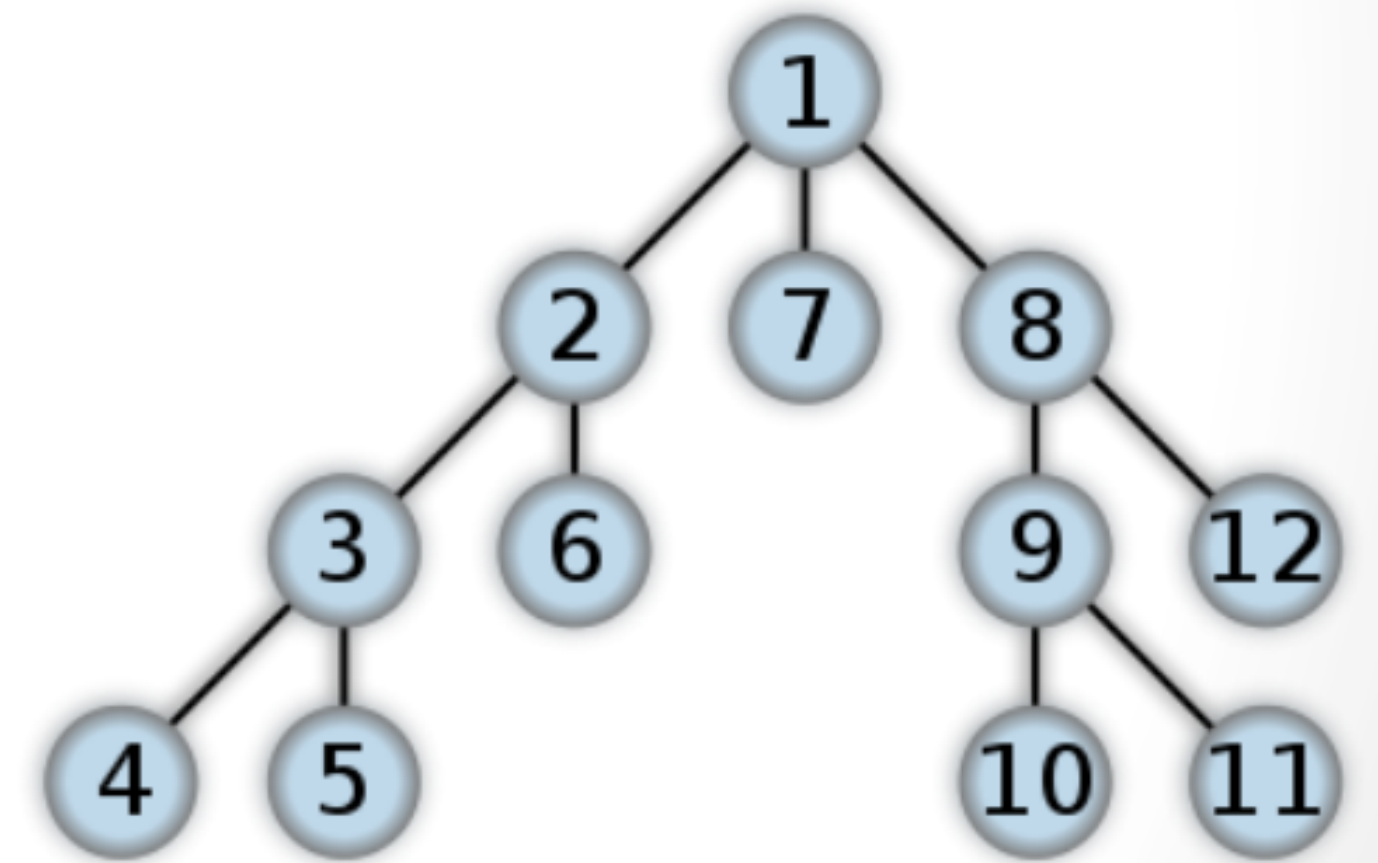
```
DFS (G, v):  
    initialize an empty stack S  
    for each vertex u: set visited[u] = false  
    push v to S  
    while (S is not empty):  
        u = pop S  
        print u  
        if visited[u] is false:  
            visited[u] = true  
            for each unvisited neighbor w of u, push w to S
```



BFS uses a queue

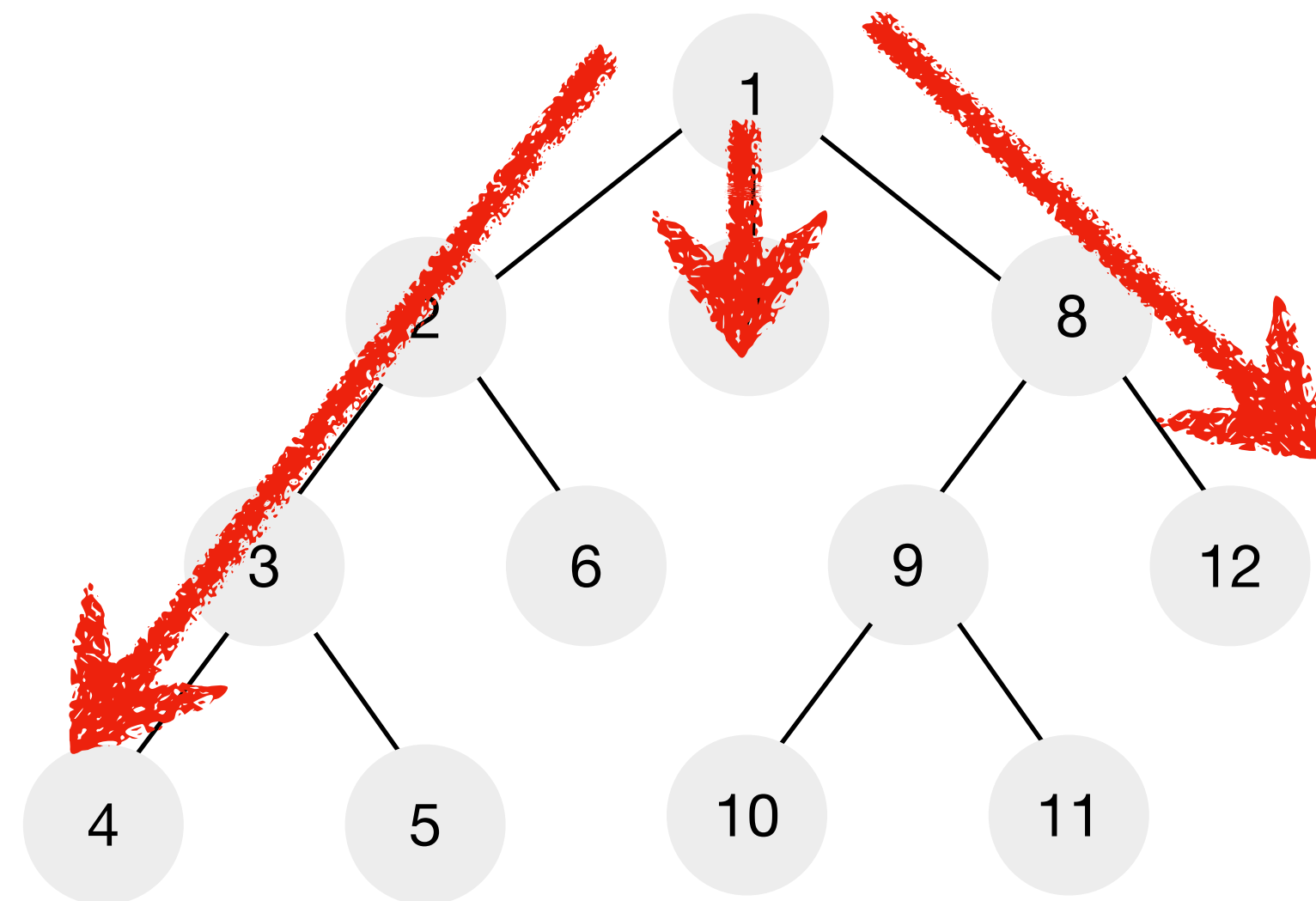
```
/* G is the graph (or tree) to search  
v is the vertex where the search starts */
```

```
BFS (G, v):  
    initialize an empty queue Q  
    for each vertex u: set visited[u] = false  
    enqueue v to Q  
    while (Q is not empty):  
        u = dequeue Q  
        print u  
        if visited[u] is false:  
            visited[u] = true  
            for each unvisited neighbor w of u, enqueue w to Q
```

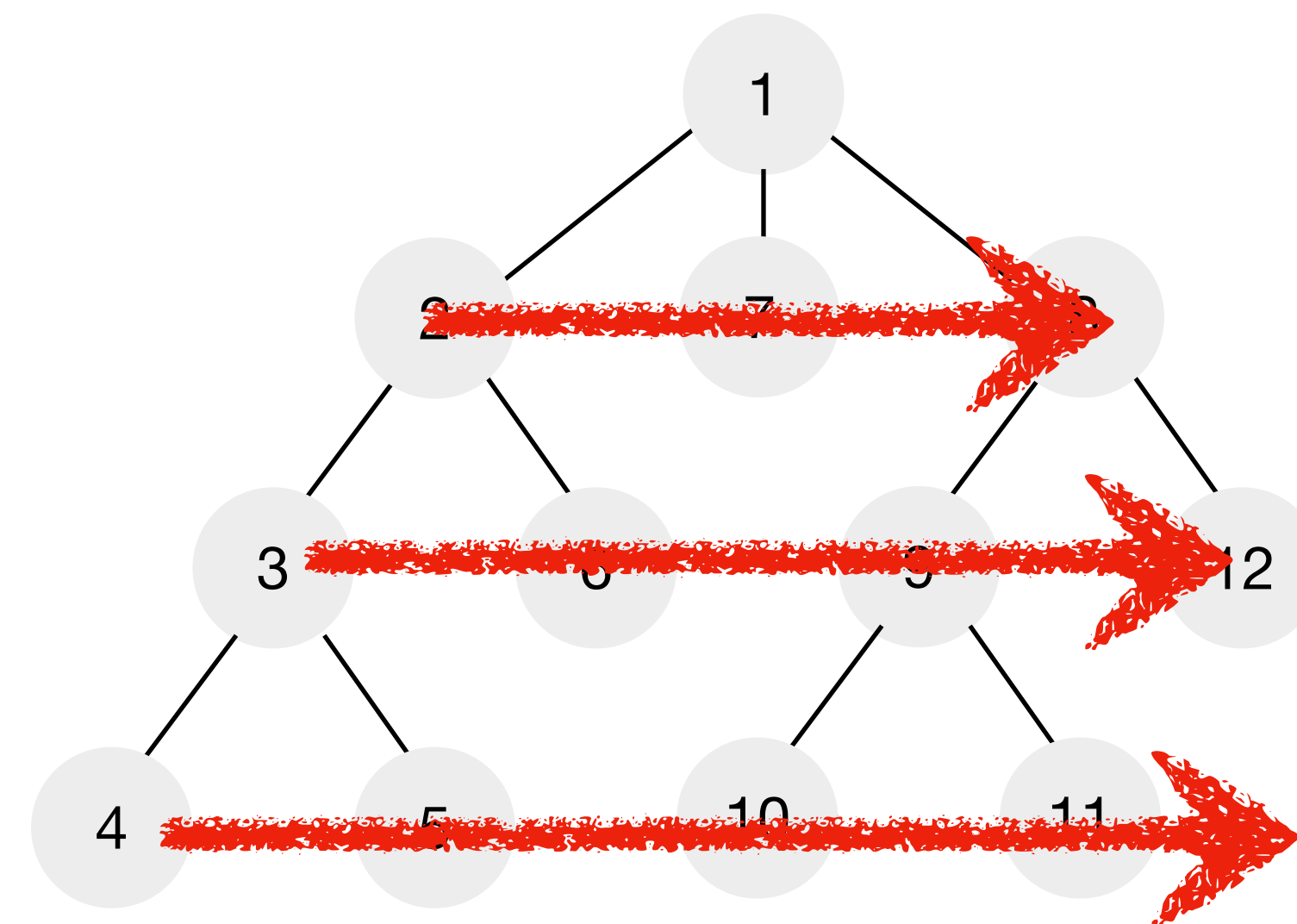


Graph Traversal

Depth First



Breadth First



- Which is faster?

class[Buzz]: graph_traversal

Time complexity of DFS and BFS is $O(|V|+|E|)$

```
/* G is the graph (or tree) to search  
v is the vertex where the search starts */
```

```
BFS (G, v):  
    initialize an empty queue Q  
    for each vertex u: set visited[u] = false  
    enqueue v to Q  
    while (Q is not empty):  
        u = dequeue Q  
        print u  
        if visited[u] is false:  
            visited[u] = true  
            for each unvisited neighbor w of u, enqueue w to Q
```

DFS vs. BFS summary

- when you know the searching key is closer to the root, it's better to use BFS
- if you know the searching key is located deep in the tree, it's better to use DFS
- if you have memory constraint, it's better to use BFS

Final Project

- Released today, and due Wednesday December 10th at 11:59pm
- Be sure to read the questions and point break-down
- All the data you need is available on DataHub in the 'public' folder
- Question 1 provides an optional opportunity to makeup for upto 20% of the midterm grade. Your mid-term grade can't go above 100% though.
- Let's take a look...